LAB – 1 Floating-Point Multiplication

Prepared by

Student name: Hadi Beiram Student ID: 300299908

Student name Hao Sun Student ID: 300226096

Group #: A0X- 3

CEG 3156: Computer Systems Design (Winter 2026)

Prof. Rami Abielmona

Presented to: Youssef Elfaramawy & Ro'yah Radaideh

TA and Lab Supervisor:

uOttawa

School of Electrical Engineering and Computer Science

École de science informatique et de génie électrique

Faculty of Engineering | Faculté de génie

University of Ottawa | Université d'Ottawa

Submission Date: February 11, 2026

Table of content

LAB – 1 Floating-Point Multiplication 1

1. Introduction 4

1.1 Summary 4

1.2 Background Information 5

1.2.1 16-bit floating-point format (custom) 5

1.2.2 Floating-point addition: alignment → add/subtract → normalization 5

1.2.3 Floating-point multiplication: sign XOR → exponent add (minus bias) → mantissa multiply → normalization 6

1.2.4 Why a Control Path/Data Path architecture is used 6

2. Design 7

2.1 Design methodology (RTL + Control/data Path split) 7

2.2 Top-level integration 8

2.3 Data Path implementation 8

2.3.1 Input operand registers 8

2.3.2 Exponent and mantissa comparators 8

2.3.3 Shifting for alignment and normalization 9

2.3.4 Mantissa add/subtract core using complementors 9

2.3.5 Output registers and result capture 10

2.3.6 Datapath status flags.....10

2.4 Control Path FSM (exact states & signals from Control Path. vhd).....10

2.4.1 S0 — Load operands 11

2.4.2 S1 / S2 — Exponent alignment (two directions).....11

2.4.3 S3 — Latch the result exponent 12

2.4.4 S4 / S5 / S6 — Select add vs subtract and determine sign handling 12

2.4.5 S7 / S8 — Select which operand to complement (invB / invA).....13

2.4.6 S9 — Complement output mantissa if required (invMO).....13

2.4.7 S10 — Normalization loop (shift mantissa and correct exponent).....13

2.5 How this maps to the lab algorithm 14

2.6 Multiplier design 14

3. Simulation & Results 15

3.1.1 Adder 15

3.1.2 Multiplier.....16

4. Conclusion 16

5. Appendices 18

5.1.1 Pre-lab 18

5.1.2 Lab Book (CEG 3156 Lab 1) 19

5.1.3 Lab Code Zip.....19

5.1.4 Graphs 19

5.1.5 GitHub.....23

1. Introduction

1.1 Summary

Floating-point arithmetic is fundamental in digital systems because it supports a wide dynamic range while maintaining reasonable precision. In this lab, we designed and implemented a custom 16-bit floating-point data path and controller to perform floating-point addition and floating-point multiplication. The 16-bit format consists of a 1-bit sign, a 7-bit exponent using excess-63 (bias = 63), and an 8-bit mantissa, as specified in the lab manual. Like $(-1)^S \times M \times 2^E$

The design follows a structured RTL methodology with a clear separation between the Data Path (registers, comparators, adders, shifters, and mantissa/exponent processing logic) and the Control Path (finite state machine generating load/shift/increment and select signals). The arithmetic operations are completed over multiple clock cycles using synchronous control, enabling step-by-step alignment/normalization and overflow handling.

To validate correctness and timing behavior, we developed test vectors covering typical and edge cases (like, sign combinations, normalization scenarios, and

overflow conditions) and verified the outputs using functional and timing simulations. This report describes the floating-point representation, algorithmic flow for addition and multiplication, the overall architecture and control strategy, and the simulation results that demonstrate compliance with the required specifications.

.

1.2 Background Information

1.2.1 16-bit floating-point format (custom)

This lab uses a custom 16-bit floating-point representation (not IEEE-754). The 16 bits are partitioned into:

- **Sign (S)**: 1 bit
- **Exponent (E)**: 7 bits, stored in **excess-63 (bias = 63)**
- **Mantissa (M)**: 8 bits

A value is interpreted as:

$$V = (-1)^S \times (1.F) \times 2^{(E-63)}$$

where $1.F$ denotes a normalized significand with an implicit leading 1, and $E - 63$ is the unbiased exponent. Using a biased exponent simplifies exponent comparison and exponent arithmetic in hardware.

1.2.2 Floating-point addition: alignment → add/subtract → normalization

Floating-point addition cannot directly add mantissas unless both operands share the same exponent. Therefore, addition typically follows these steps:

- **Exponent comparison and alignment**
  Compare E_A and E_B. Right-shift the mantissa of the operand with the smaller exponent until the exponents match. This preserves the numerical value while aligning binary points.
- **Mantissa add/subtract based on sign**
  If $S_A = S_B$, the mantissas are added; if $S_A \neq S_B$, the mantissas are effectively subtracted (larger magnitude minus smaller magnitude), and the output sign follows the operand with the larger magnitude.
- **Normalization and exponent update**
  After addition/subtraction, the result mantissa may be outside the normalized range

(e.g., carry-out after addition or leading zeros after subtraction). The result is normalized by shifting and adjusting the exponent accordingly.

- **Overflow detection**
  If the exponent exceeds the representable range after normalization, an overflow condition is asserted.

1.2.3 Floating-point multiplication: sign XOR → exponent add (minus bias) → mantissa multiply → normalization

Floating-point multiplication generally proceeds as:

- **Sign computation**

  $S_{out} = S_A \oplus S_B$

- **Exponent computation**
  Because both exponents are biased, multiplying requires adding exponents and subtracting the bias once:

  $E_{out} = E_A + E_B - 63$

- **Mantissa multiplication**
  The significands are multiplied, producing a wider product than the 8-bit mantissa field. The most significant bits of the product determine whether the result is already normalized.

- **Normalization and exponent correction**
  If the product is too large (e.g., leading bits indicate ≥ 2), the mantissa is right-shifted and the exponent is incremented; if the product is too small (implementation-dependent in this lab scope), normalization shifts and exponent adjustment are applied.

- **Overflow detection**
  If the resulting exponent exceeds the 7-bit limit after normalization, the overflow output is asserted.

1.2.4 Why a Control Path/Data Path architecture is used

Both addition (alignment/normalization) and multiplication (normalization/overflow handling) may require iterative shifting and conditional updates. A common lab-friendly hardware approach is to separate:

- **DataPath**: registers + arithmetic/shift/compare blocks that produce intermediate values and flags

- **ControlPath (FSM/ASM)**: a controller that sequences the operation over multiple clock cycles by issuing control signals (load, shift, increment, select) based on datapath flags

This structure makes the design modular, debuggable, and consistent with synchronous digital design practices.

- Multimeter
- Laboratory Power Supply
- Oscilloscope
- Resistors
- Variable Resistors (Potentiometers)
- Bread Board
- Software: Multisim 14.1

2. Design

2.1 Design methodology (RTL + Control/data Path split)

This project is implemented using RTL, modular methodology. The top-level schematic (lab1CEG3156.bdf) instantiates two main blocks: ControlPath and DataPath. The DataPath contains registers and combinational logic (comparators, adders, shifters, complementors and multiplexers) to manipulate mantissas/exponents and to generate status flags. The ControlPath is a multi-cycle controller implemented as a one-hot FSM (11 state flip-flops ffOuts (0..10)) that sequences the floating-point algorithm by asserting load/shift/increment/invert control signals based on data path flags.

2.2 Top-level integration

At the top level, the external I/O ports are the lab-required fields: GClock, GReset, SignA, ExponentA[6..0], MantissaA[7..0], SignB, ExponentB[6..0], MantissaB[7..0], and the outputs SignOut, ExponentOut[6..0], MantissaOut[7..0] (and overflow if implemented). The top-level wires the datapath flags back into the controller (e.g., EAgtEB, EBgtEA, EAeqEB, SAeqSB, MAgtMB, MO7, SO, SA) and wires controller outputs into the data path control inputs (e.g., loadMA, shiftMA, incEA, etc.).

2.3 Data Path implementation

The data Path is responsible for storing the operands, performing combinational arithmetic, and producing status flags that guide the controller. In our implementation (based on DataPath.bdf), the data path is built from synchronous registers and basic RTL “atomic” blocks (comparators, adders, shifters, complementors, and multiplexers), consistent with the lab’s structural design requirements.

2.3.1 Input operand registers

The fields of each land are latched into internal registers under explicit load control. Specifically, the sign, exponent, and mantissa for operand A (SA, EA, MA) and operand B (SB, EB, MB) are captured using the control signals loadSA, loadEA, loadMA, loadSB, loadEB, and loadMB. Registering the inputs ensures fully synchronous operation and prevents glitches from propagating into the arithmetic path.

2.3.2 Exponent and mantissa comparators

The data path includes comparator8_bit blocks to generate comparison flags used for decision-making. Exponent comparisons produce EAgtEB, EBgtEA, and EAeqEB

to support alignment (i.e., determining which operand has the smaller exponent). Mantissa comparison flags (e.g., MAgtMB, MAeqMB, MBgtMA) are used in opposite-sign cases to determine relative magnitude and therefore the subtraction order and final sign selection.

2.3.3 Shifting for alignment and normalization

Alignment is implemented through controlled right-shifting of the mantissa of the operand with the smaller exponent. When A has the smaller exponent, shiftMA is asserted and incEA updates A's exponent toward the larger exponent. Similarly, when B has the smaller exponent, shiftMB and incEB are asserted. After the mantissa operation, normalization support is provided through the output mantissa shift control shiftMO together with exponent correction via incEO. This enables iterative normalization in multi-cycle operation.

2.3.4 Mantissa add/subtract core using complementors

The mantissa arithmetic is implemented with an fullAdder8_bit block for addition. Subtraction is realized using two's-complement conversion so that subtraction can be performed by the same adder. The complementor8_bit block generates the two's complement of a selected operand, controlled by invA and/or invB, and the data path uses MUX2_8bit selection (controlled by signals such as SMUX) to route either the original mantissa or the complemented version into the adder. In some cases, the result mantissa may also require correction via the output complement control invMO.

2.3.5 Output registers and result capture

Results are stored in dedicated output registers to ensure stable synchronous outputs. The controller asserts loadSO to capture the output sign, loadEO to capture the aligned/updated exponent, and loadMO to capture the mantissa result. This register-based output stage makes the top-level outputs (SignOut, ExponentOut, MantissaOut) transition only on clock edges.

2.3.6 Datapath status flags

To support the multi-cycle control strategy, the data path exports key flags back to the controller. For example, EAgtEB/EBgtEA/EAeqEB indicate exponent alignment status, SAeqSB indicates whether the operation is pure addition or an effective subtraction, MAgtMB supports magnitude-based decisions in opposite-sign cases, and MO7 indicates whether the mantissa has reached the required normalized condition during the normalization loop.

2.4 Control Path FSM (exact states & signals from Control Path. vhd)

TheControl Path is implemented as a one-hot finite state machine (FSM) using 11 state flip-flops ffOuts(0..10) in Control Path. vhd. Each flip-flop corresponds to one state (denoted here as S0–S10). State transitions are driven by data path status flags such as exponent comparison results (EAgtEB, EBgtEA, EAeqEB), sign relationship (SAeqSB), magnitude comparison (MAgtMB), and normalization conditions (MO7, SO, SA). The controller generates the data path control signals (e.g., load, shift, increment, invert, and mux select) to execute the floating-point operation across multiple clock cycles.

2.4.1 S0 — Load operands

In S0, the controller asserts the operand load enables:

loadMA, loadEA, loadSA, loadMB, loadEB, loadSB

This captures A and B (sign, exponent, mantissa) into internal datapath registers, ensuring synchronous operation.

2.4.2 S1 / S2 — Exponent alignment (two directions)

The FSM then performs exponent alignment based on the exponent comparator flags:

If EAgtEB = 1, the machine enters S1 and asserts:

incEB = 1 and shiftMB = 1

This shifts B's mantissa right while incrementing EB until EB matches EA.

If EBgtEA = 1, the machine enters S2 and asserts:

incEA = 1 and shiftMA = 1

This shifts A's mantissa right while incrementing EA until the exponents match.

When EAeqEB = 1, exponent alignment is complete and the controller advances to the next stage.

2.4.3 S3 — Latch the result exponent

In S3, the controller asserts:

loadEO = 1

The aligned exponent value is captured into the output exponent register path.

2.4.4 S4 / S5 / S6 — Select add vs subtract and determine sign handling

After alignment, the operation depends on whether the operand signs are equal:

- S4 (same sign): when `SAeqSB = 1`

  The controller selects the direct addition path (via `SMUX`) and captures the sign/mantissa result:

  `SMUX = 1`, `loadSO = 1`, `loadMO = 1`

  Conceptually, this corresponds to mantissa addition with the output sign matching the common operand sign.

- S5 / S6 (opposite sign): when `SAeqSB = 0`

  The datapath must perform an effective subtraction (larger magnitude minus smaller magnitude). The FSM branches based on mantissa magnitude comparison (`MAgtMB`) to decide subtraction order and sign selection. In these states, the controller prepares the datapath for two's-complement-based subtraction and captures the output sign using:

  `loadSO = 1`

  while subsequent states control which operand is complemented.

2.4.5 S7 / S8 — Select which operand to complement (invB / invA)

To realize subtraction using the same adder, the design uses two's-complement conversion:

- S7 asserts invB = 1 to complement operand B's mantissa before the add operation.
- S8 asserts invA = 1 to complement operand A's mantissa before the add operation.

The controller selects between S7 and S8 based on the sign/magnitude conditions (including SA and the magnitude comparison), ensuring the datapath performs "larger − smaller" correctly.

2.4.6 S9 — Complement output mantissa if required (invMO)

In some subtraction/sign combinations, the intermediate mantissa result may require an additional correction. In S9, the controller asserts:

2.4.7 S10 — Normalization loop (shift mantissa and correct exponent)

The controller normalizes the mantissa result. In S10, it repeatedly asserts:

`shiftMO = 1` and `incEO = 1`

The FSM remains in this normalization loop until the datapath indicates a normalized condition via:

`MO7 = 1`

Once `MO7` confirms the mantissa has reached the required normalized form, the FSM exits and the computation is complete.

Overall, the control Path sequences the data path through: operand load → exponent alignment → mantissa add/subtract selection → (two's complement steps for subtraction) → optional output correction → normalization. The one-hot FSM structure (ffOuts(0..10)) makes control signal generation straightforward, and the data path feedback flags ensure correct conditional branching and multi-cycle iteration.

2.5 How this maps to the lab algorithm

Alignment (S1/S2) implements the standard step "shift the smaller mantissa until exponents match". (ceg3156Lab1)

Mantissa add/sub selection (S4–S8) implements the sign-dependent add/sub behavior and magnitude selection.

Normalization (S10) implements the "normalize if necessary" step of floating-point arithmetic. (ceg3156Lab1)

2.6 Multiplier design

According to the lab manual, the floating-point multiplier follows a three-step algorithm: (1) add the exponents and subtract the bias, (2) multiply the mantissas and determine the sign, and (3) normalize the result if needed.

Input/Output format

Both the adder and multiplier use the same 16-bit floating-point format: 1-bit sign, 7-bit exponent (excess-63), and 8-bit mantissa, with synchronous `GClock` and global `GReset`, and an `Overflow` output to indicate overflow detection.

Step 1 — Exponent computation (excess-63)

The multiplier computes the output exponent by adding the two biased exponents

and subtracting the bias once, consistent with the manual's "add exponents and subtract bias" rule.

Step 2 — Sign and mantissa product

The output sign is determined from the operand signs (equivalent to XOR behavior). The mantissas are multiplied to produce a product wider than 8 bits, because multiplying two normalized significands generates additional product bits. The datapath then selects the correct normalized mantissa field for `MantissaOut [7:0]`.

Step 3 — Normalization and overflow

After multiplication, the product may require normalization. The design checks the leading product bit(s) and performs a shift, if necessary, with a corresponding correction to the output exponent, as required by the manual's "normalize if necessary" step.

Finally, the design asserts `Overflow` if the resulting exponent exceeds the representable range for the 7-bit exponent field.

3. Simulation & Results

3.1.1 Adder

Using `GClock` and `GReset`, the one-hot FSM `ffOuts (0..10)` sequences alignment → mantissa add/sub → normalization, and registers final outputs using `loadSO/loadEO/loadMO`.

Key observed signals

include `EAgtEB/EBgtEA/EAeqEB`, `shiftMA/shiftMB`, `incEA/incEB`, `SAeqSB`, `invA/invB/invMO`, and normalization flags such as `MO7` with `shiftMO/incEO`.

3.1.2 Multiplier

Sign check: simulation confirms SignOut = SignA XOR SignB.

Exponent check: simulation confirms exponent processing follows

$E_{out} = E_A + E_B - 63$ (excess-63 bias).

Mantissa/product + normalization: mantissas are multiplied to form a wider product; the waveform verifies normalization is applied (conditional shift of the product and corresponding exponent adjustment) until the output mantissa fits the 8-bit field.

Timing simulation: a timing simulation screenshot is included as required to confirm clocked stability of outputs and correct propagation delays.

4. Conclusion

The design approach follows the lab's structured RTL expectation: multi-step floating-point operations are executed over multiple cycles with registered I/O and clear control

Adder: alignment is necessary before mantissa arithmetic; the observed alignment loop (shift + exponent adjustment until exponents match) and the normalization loop are consistent with the standard floating-point addition flow.

Multiplier: the implementation follows the required multiplication flow (Fig. 6): sign XOR, biased-exponent add with bias subtraction, mantissa multiplication, and normalization with exponent correction.

Trade-offs: multi-cycle control reduces combinational complexity and makes

debugging easier (waveforms show step-by-step progression) but increases latency

for large exponent differences (adder) or repeated normalization (multiplier).

Limitations:

If guard/round/sticky bits are not implemented, right shifts and truncation may introduce rounding error; IEEE special values are outside this simplified 16-bit format.

Simplified rounding: If guard/round/sticky bits are not implemented, right shifts during exponent alignment (adder) and normalization (adder/multiplier) may cause truncation error and reduced precision.

Variable latency: The multi-cycle approach introduces data-dependent latency: the adder may take more cycles when the exponent difference is large, and the multiplier may take more cycles when normalization requires repeated shifting.

Overflow emphasis, underflow not detailed: Overflow detection is typically included, while detailed underflow behavior may be simplified depending on scope.

Output waveform may look “incorrect” or unstable: After reset is released, SignOut/ExponentOut/MantissaOut can change for several cycles before settling. This is expected in a multi-cycle datapath, where intermediate stages (alignment/operation/normalization and register updates) temporarily affect the output before the final write-back result becomes stable.

5. Appendices

5.1.1 Pre-lab

Usually, a pre-lab would be associated with each laboratory, but due to time constraints, this experiment will not require a pre-lab. Successive experiments, however, will have a pre-lab section that has to be completed by the group before attending the allocated laboratory hours.

5.1.2 Lab Book (CEG 3156 Lab 1)

5.1.3 Lab Code Zip

5.1.4 Graphs

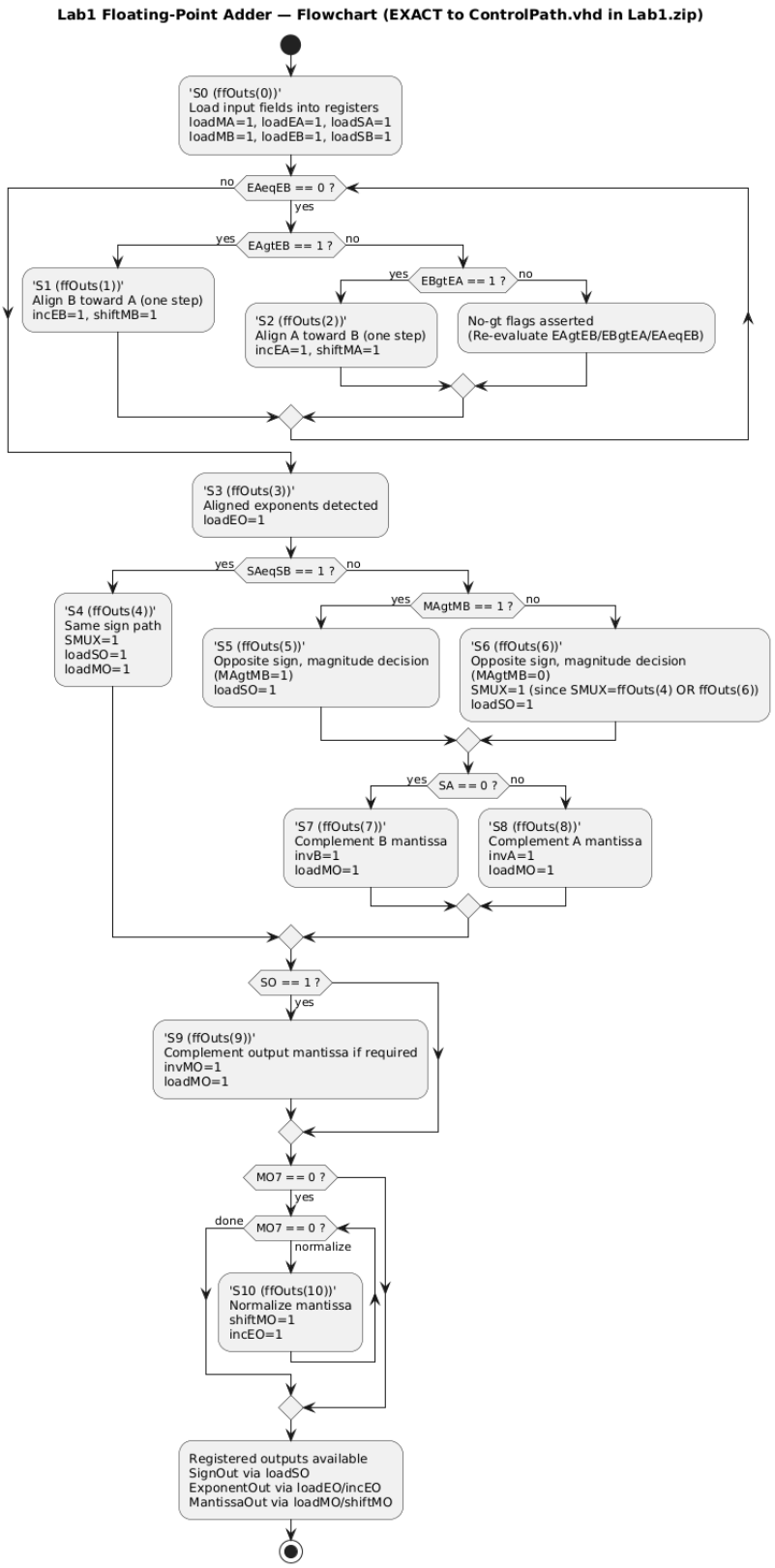


Figure 1:Lab1 Floating-Point Adder — Flowchart

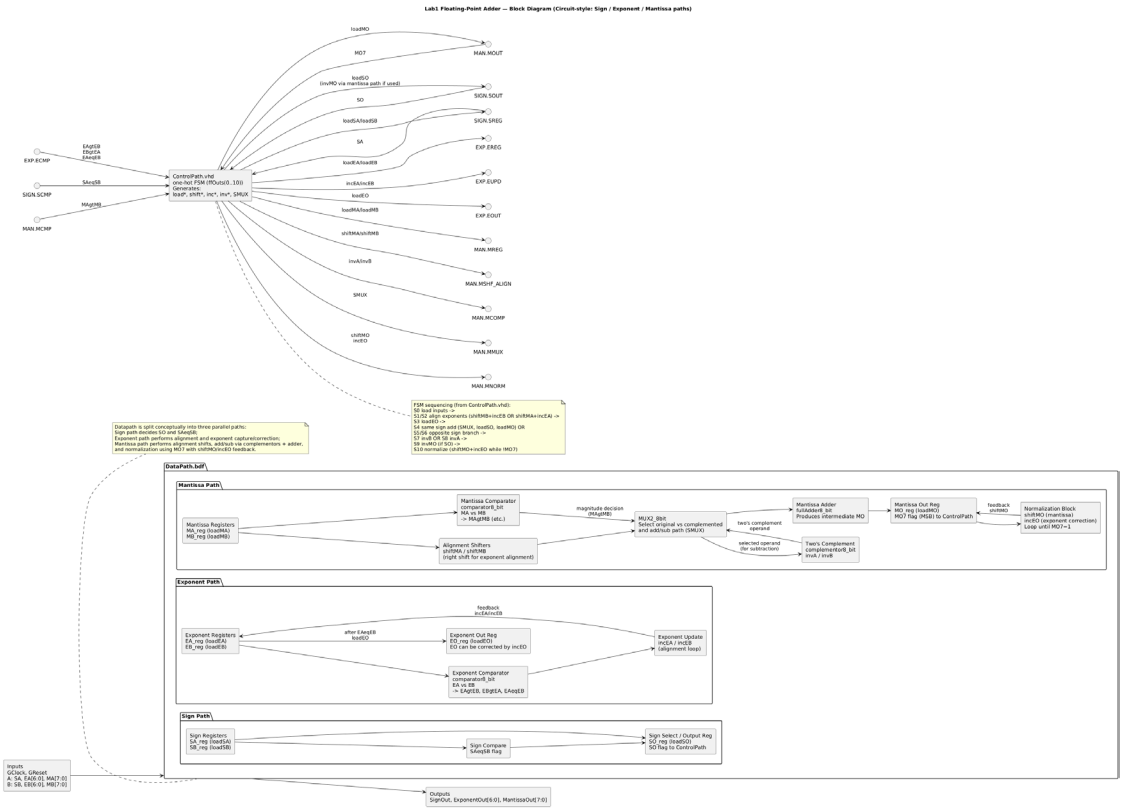


Figure 2: Floating-Point Adder — Block Diagram

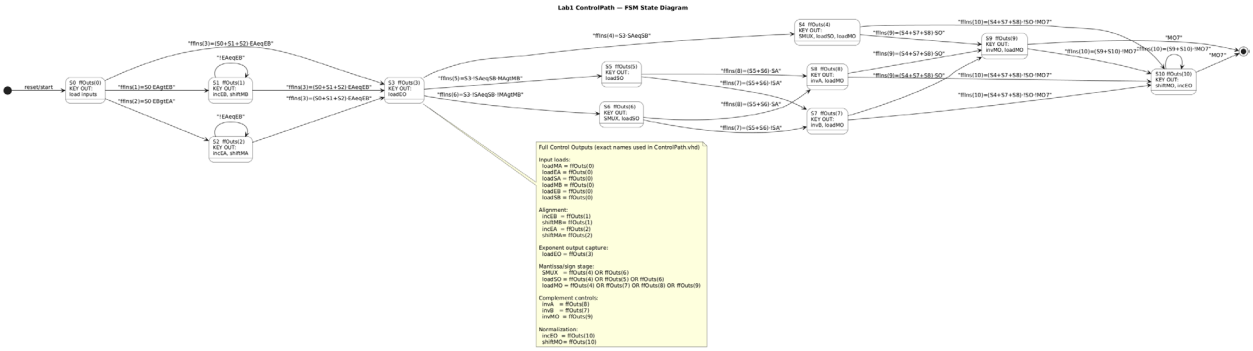
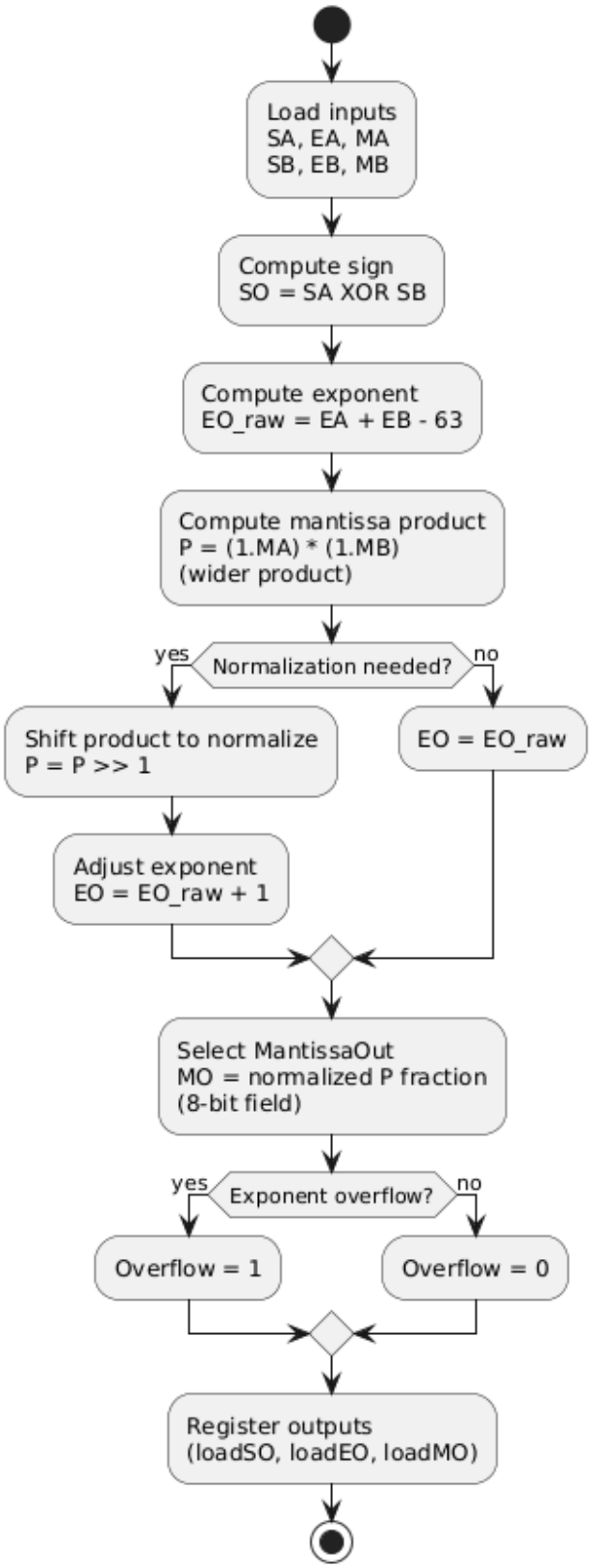
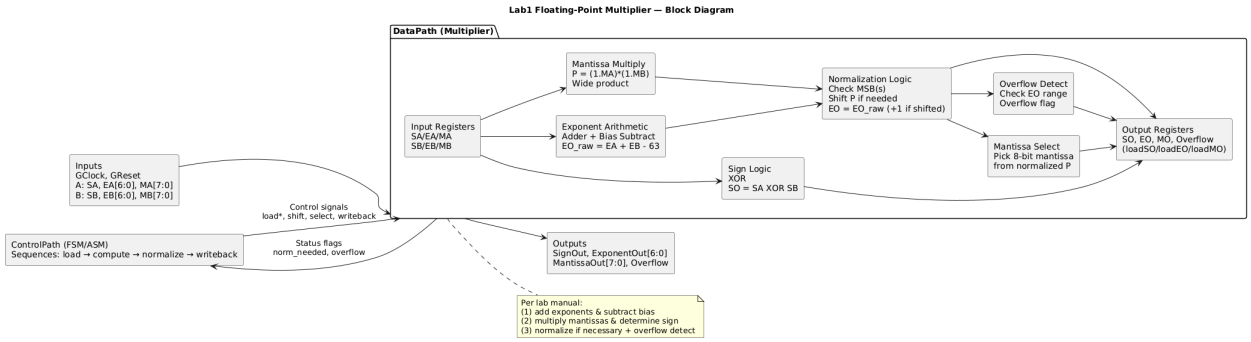
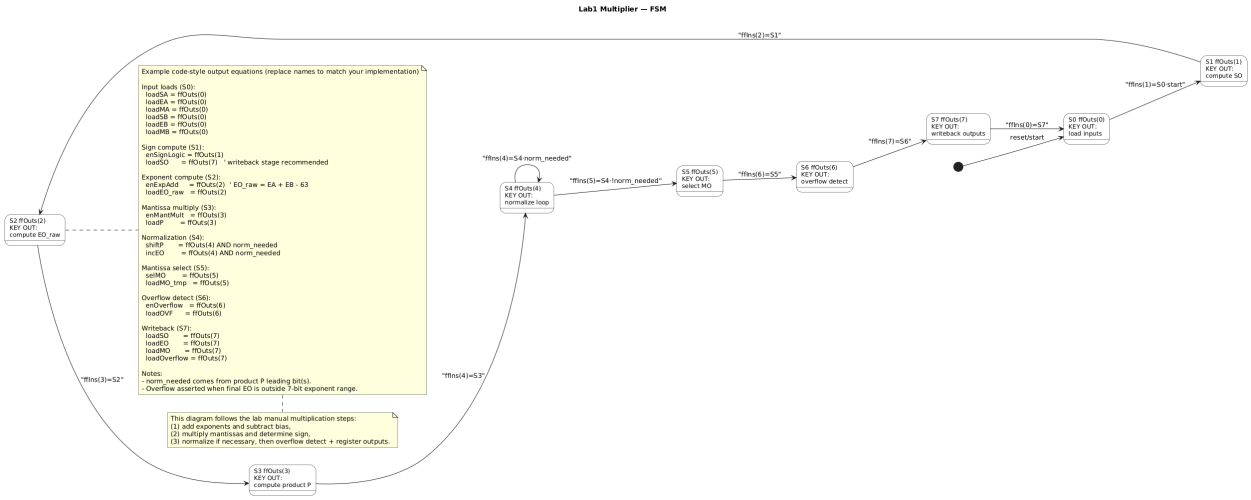


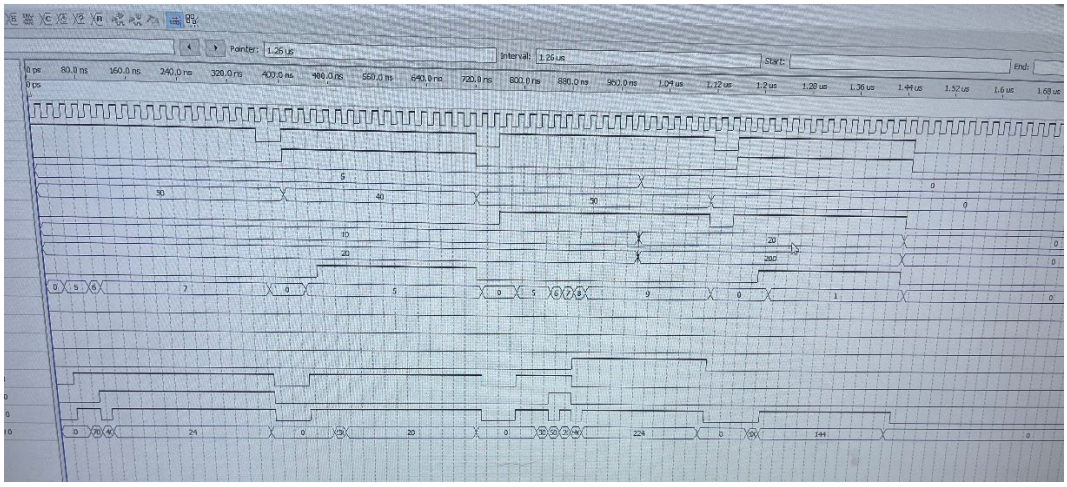
Figure 3: FSM State Diagram

Lab1 Floating-Point Multiplier — Flowchart









5.1.5 GitHub

https://github.com/xuanwu12345/ceg-3156-group-a0x-3.git